

Conserta Bairro

Modelagem de um sistema de informação em formato PWA

Entrega 1 - Práticas Extensionistas IV

Integrantes

Bernardo Haro Massignani

Marcelo Schuermann

Repositório da disciplina

<https://github.com/akaharo/praticas-extensionistas-iv>

Proposta

O Conserta Bairro aproxima moradores que têm objetos domésticos com pequenos defeitos de voluntários e espaços comunitários que podem ajudar no reparo. O pedido informa categoria, descrição e bairro. Um voluntário aceita o pedido, combina um ponto de encontro e atualiza o andamento. O objetivo é facilitar o reaproveitamento no bairro.

Escopo desta entrega

Este documento apresenta os três diagramas pedidos e a escolha da infraestrutura. A aplicação e os serviços em nuvem ainda não foram implantados; os fluxos mostram a arquitetura prevista para a etapa de desenvolvimento.

Curso

Análise e Desenvolvimento de Sistemas

Instituição

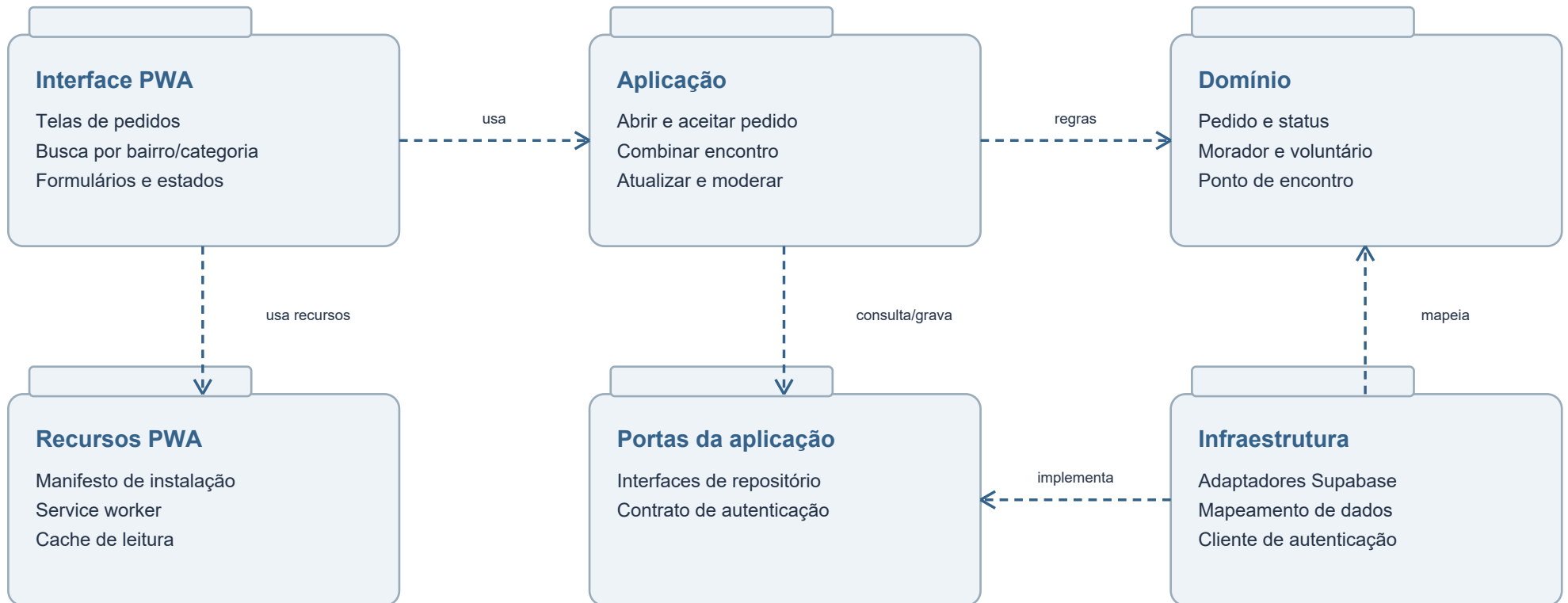
UNOESC

Professor

Jean Carlos Hennrichs

1. Diagrama UML de pacotes

Dependências entre as partes planejadas da aplicação

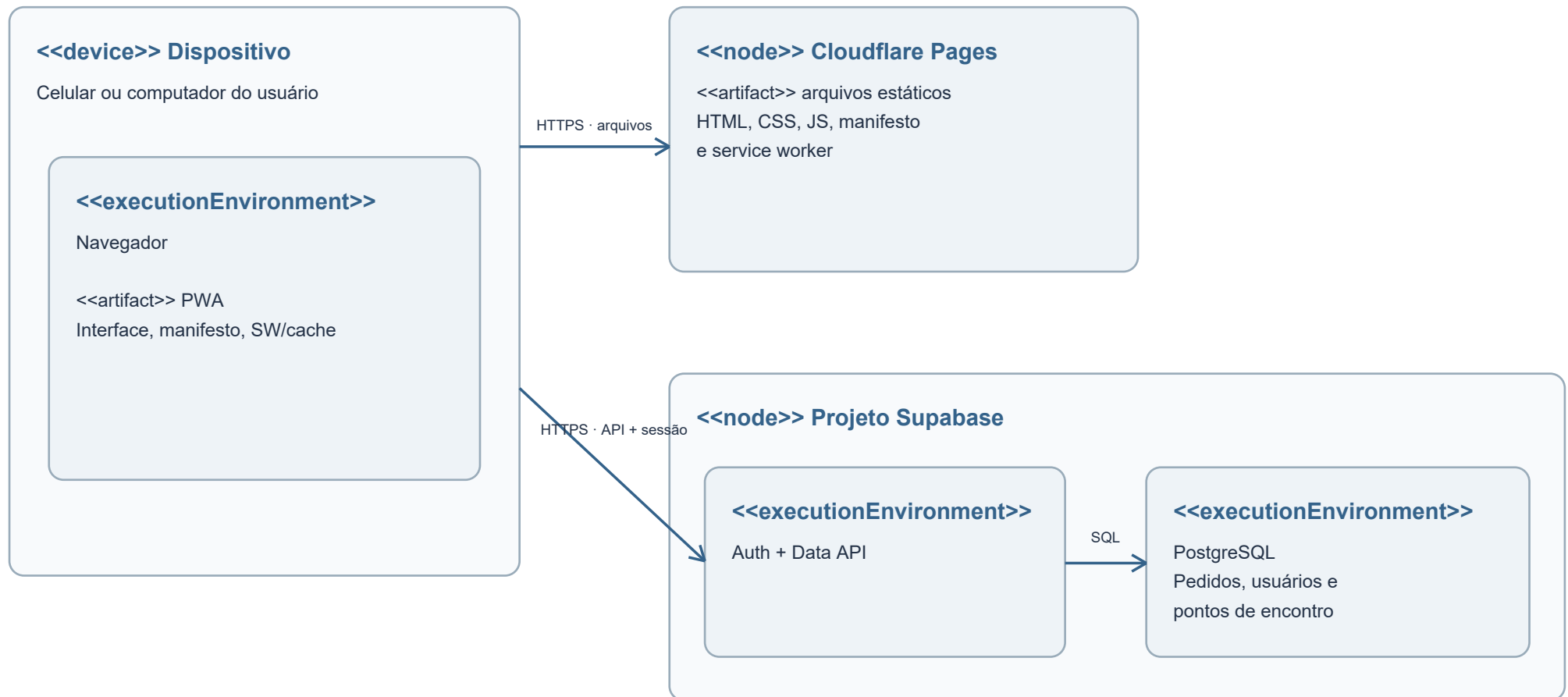


Seta tracejada: o pacote na origem depende do pacote apontado.

O cache permite consultar dados já carregados; criar ou alterar pedidos depende de conexão.

2. Diagrama UML de implantação

Nós, ambientes de execução, artefatos e comunicação

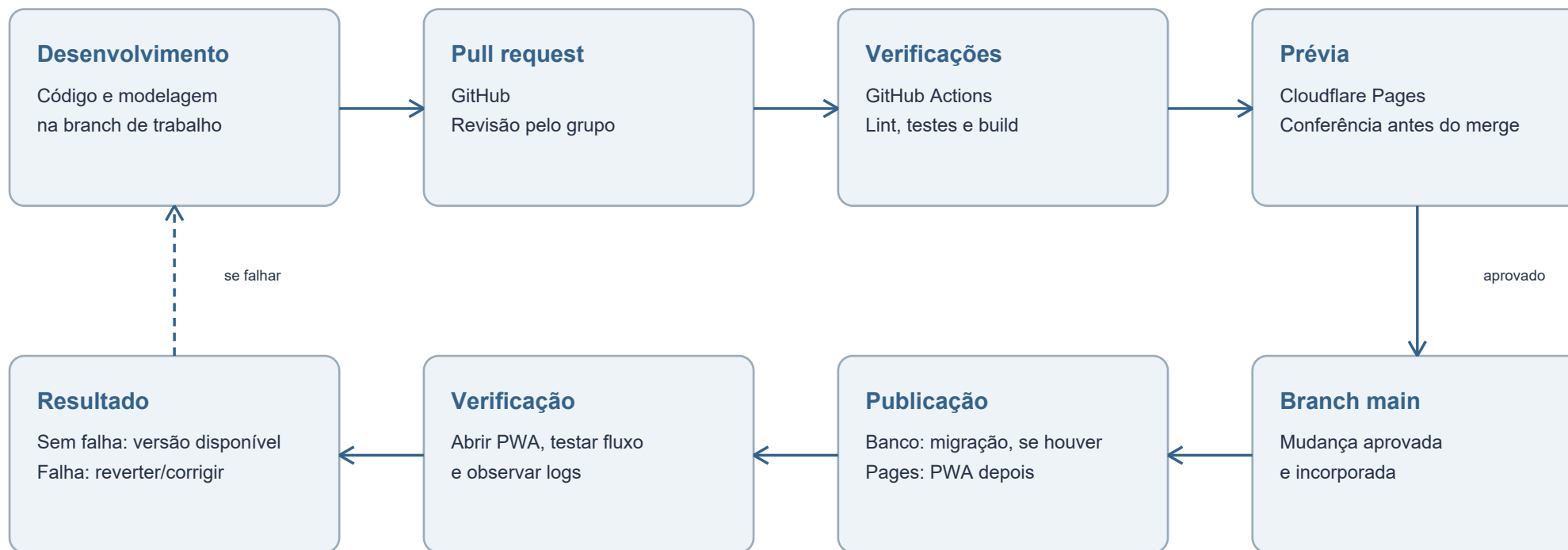


O cache ajuda a ler a lista pública sem conexão. Os pedidos atualizados ficam no Supabase.

RLS limita o acesso aos registros; dados de contato ficam restritos aos envolvidos no pedido.

3. Arquitetura DevOps

Fluxo planejado para quando a aplicação for desenvolvida



Mudanças no banco são aplicadas antes do cliente que depende delas. Em caso de falha, o grupo corrige a versão.

Este é o processo proposto; ainda não há pipeline configurada ou aplicação publicada.

4. Infraestrutura de publicação

Escolha e justificativa para a primeira versão

Escolha: Cloudflare Pages + Supabase

O Pages hospedar  os arquivos est ticos da PWA em HTTPS. O Supabase oferecer  login, API e PostgreSQL para pedidos, usu rios e pontos de encontro. A combina  o evita manter um servidor pr prio para um projeto pequeno e mant m os dados separados dos arquivos da interface.

Alternativas consideradas

Servidor pr prio/VPS: mais controle, por m exige configurar sistema, HTTPS, banco e backups.

Firebase: atende ao caso, mas preferimos PostgreSQL para relacionar pedidos, pessoas e locais.

S  GitHub Pages: hospeda a interface, mas n o fornece sozinho login e persist ncia dos pedidos.

Antes de publicar

Ativar as contas do grupo; criar o projeto Supabase; aplicar tabelas e pol ticas RLS; conectar o GitHub ao Cloudflare Pages; configurar vari veis p blicas; testar a PWA e o fluxo de pedidos.

Segredos administrativos n o devem entrar no cliente nem no reposit rio. O service worker guardar  a interface e a lista p blica consultada. Envio e atualiza  o exigir o conex o.

Refer ncias t cnicas

Cloudflare Pages: developers.cloudflare.com/pages/get-started/git-integration/

Supabase: supabase.com/docs/guides/platform-and-database/postgres/row-level-security

MDN PWA: developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Guides/Making_PWAs_installable

Material de Apoio Diagramas.pdf, disponibilizado na disciplina.